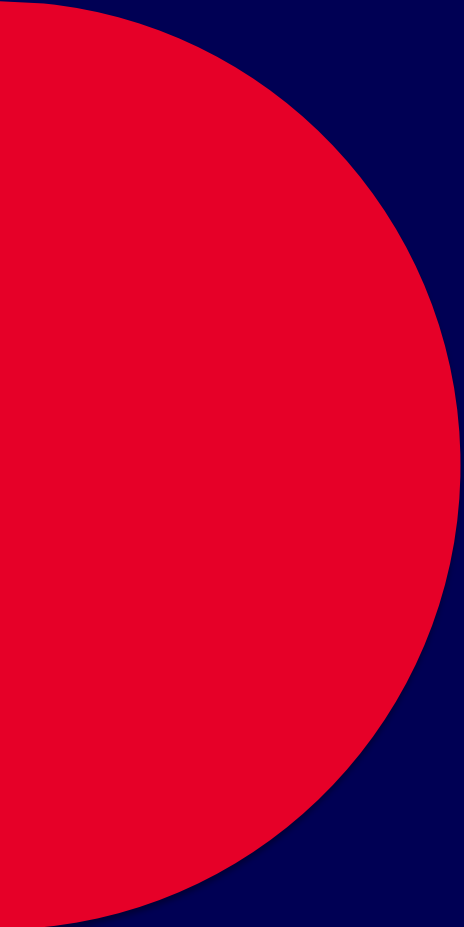




Steering Behaviours



Introduction

As discussed last week, AI for games *is all about appearances*.

We want AI-controlled characters to look as natural / realistic as possible.

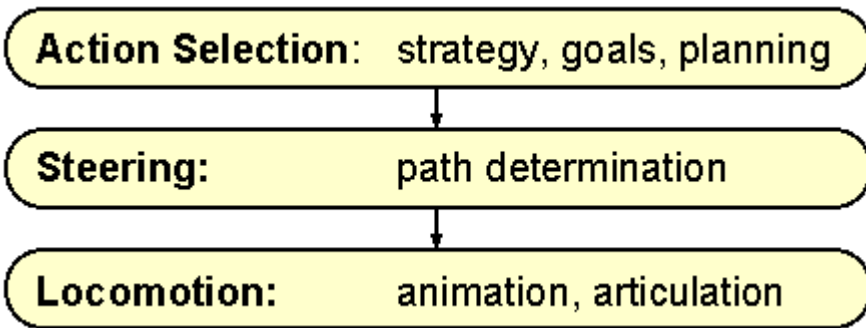
Our focus today is on **realistic movement**.



Source: <https://www.youtube.com/watch?v=6BrZryMz-ac>

Introduction

In video games there is typically a hierarchy of motion behaviors:

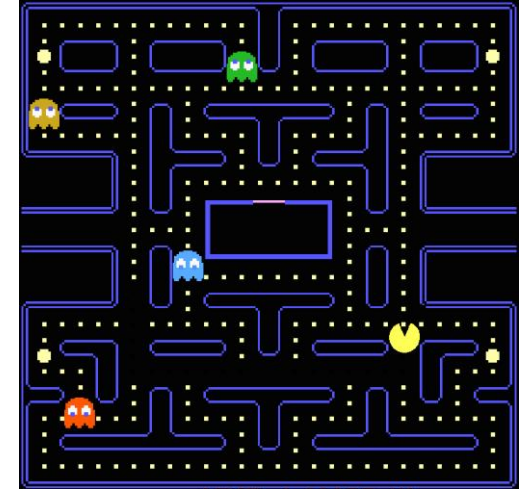


Our focus today

- High-level decision making, e.g., path planning, sits at the top level.
- Animation and articulated motion are beyond the scope of this course.

Movement

- Many old games:
 - Characters move at fixed speed, without regard to how physical objects accelerate or brake.
- Steering behaviours:
 - Characters accelerate and turn **based on physics**.
 - Lots of different behaviour types: Seek, flee, flocking...
 - Terminology introduced by Craig Reynolds in 1999.
See [Steering Behaviors For Autonomous Characters](#), linked on Canvas.



Maths / Physics Preliminaries

For this course, you should familiarise yourself with the following:

- What a **vector** is (a quantity that has both magnitude/length and direction).
- The length of vector **v** is written as $|\mathbf{v}|$.
- A **unit vector** is a vector of length 1. Typically written as $\hat{\mathbf{u}}$.
- To get a unit vector in the direction of **v**, divide by its length, i.e., $\hat{\mathbf{u}} = \mathbf{v} / |\mathbf{v}|$.
- **Velocity**: $v = dx/dt$, where x = position and t = time.*
- For an object moving at constant speed, $v = \Delta x / \Delta t$.
- **Acceleration**: $a = dv/dt$.
- Under a constant force, $a = \Delta v / \Delta t$.
- **Newton's second law**: $F = ma$.
- See the Khan Academy links on the Week 2 Canvas page if you need to brush up.

*Don't stress, we're not going to get into derivatives. Just think of velocity as "how quickly the position is changing".

Example (in 1D)

An object of mass 2kg is currently travelling at a velocity of -5m/s in the x-direction.
(Negative velocity = moving left).

A constant force is applied to the object, increasing its velocity to +10m/s after 3 seconds.
What is the magnitude of the force?

$$\Delta v = 10 - (-5) = 15\text{m/s}$$

$$a = \Delta v / \Delta t = 15 / 3 = 5\text{m/s}^2$$

$$F = ma = 2 * 5 = 10$$

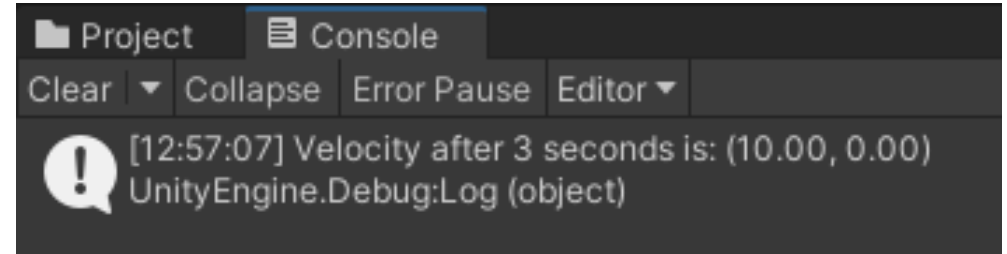
Force is measured in “Newtons” (N), so you will see this written as $F = 10\text{N}$.

Checking in Unity

```
public class NewtonDemo : MonoBehaviour
{
    Rigidbody2D _rb;

     Unity Message | 0 references
    void Start()
    {
        _rb = GetComponent<Rigidbody2D>();
        _rb.mass = 2.0f;
        _rb.velocity = new Vector2(-5.0f, 0.0f);
    }

     Unity Message | 0 references
    void FixedUpdate()
    {
        _rb.AddForce(new Vector2(10.0f, 0.0f));
        if (Time.fixedTime == 3.0f)
        {
            Debug.Log("Velocity after 3 seconds is: " + _rb.velocity);
        }
    }
}
```



Note: You will get a different result if gravity is on or linear drag is non-zero.

Steering Behaviours: General Approach

Each character has a **desired velocity**, i.e., the velocity they would like to be moving at.

- A shark chasing a fish would like to be moving towards the fish at its maximum speed.*
- The fish would like to be moving away from the shark at its maximum speed.

Key idea:

Rather than immediately setting `velocity = desiredVelocity`, gradually accelerate to `desiredVelocity`:

$$\Delta v = \text{desiredVelocity} - \text{currentVelocity}$$

$$a = \Delta v / \text{accelerationTime}$$

$$F = ma$$

$$= m * (\text{desiredVelocity} - \text{currentVelocity}) / \text{accelerationTime}$$

Steering Behaviours: General Approach

Notes:

- Acceleration is generally capped to some maximum, realistic value.
- The Assignment 1 starter code is already set up to follow the general approach described. You're expected to study it and figure out how the method calls tie together.
- Online tutorials often implicitly set $m = 1$ and $\text{accelerationTime} = 1$, which yields:

$$F = \text{desiredVelocity} - \text{currentVelocity}$$

I don't like obscuring the physics like this, and setting $\text{accelerationTime} = 1$ often feels too laggy.



Steering Behaviours: General Approach

There are many different types of steering behaviour:

- Seek, flee, arrive.
- Pursuit, evasion.
- Obstacle avoidance.
- Flocking.
- Wander, leader following, path following...

Each behaviour amounts to a different approach for calculating the desired velocity.

Behaviours can be combined to achieve various effects (as we'll see for flocking).

Seek

The **seek** steering behaviour can be paraphrased as:

“I’d like to be moving towards my target position at my maximum speed”.

Let $\mathbf{v}$ = targetPos - currentPos (i.e., a vector from the current position to the target).

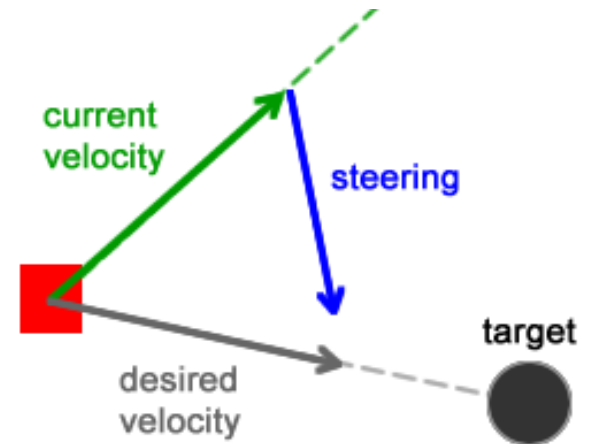
Let $\hat{\mathbf{u}}$ denote a unit vector (vector of length 1) in the direction of $\mathbf{v}$.

The desired velocity is then:

$$\text{desiredVelocity} = \hat{\mathbf{u}} * \text{maxSpeed}$$

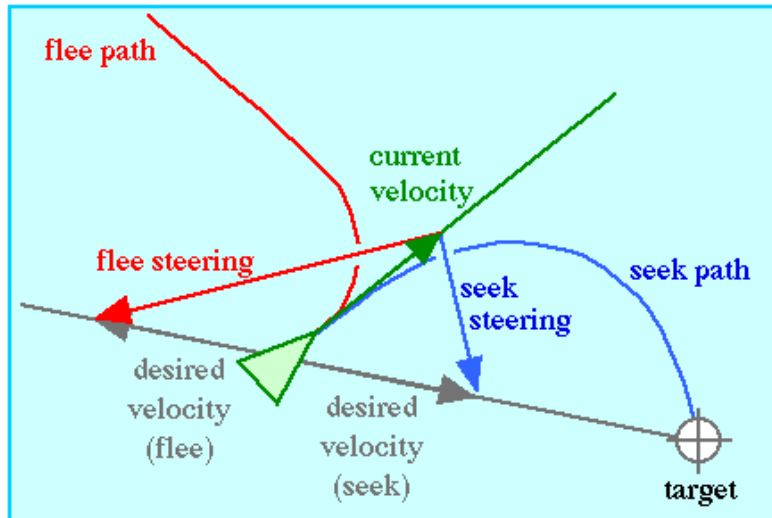
In Unity, this is:

```
desiredVelocity = (targetPos - currentPos).normalized * maxSpeed;
```



Flee

“I’d like to be moving away from the target at my maximum speed”.



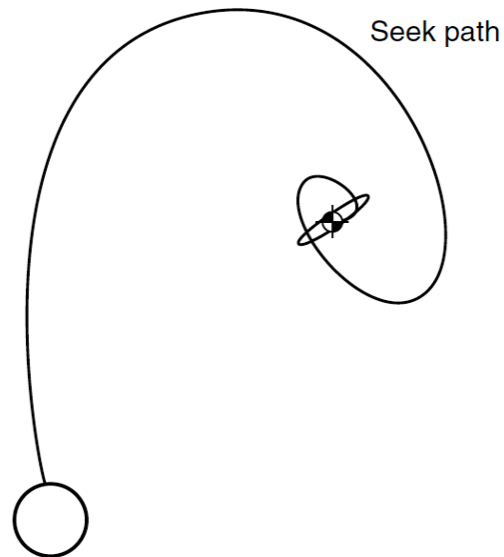
The calculation of the desired velocity is identical to **seek**, except that the resultant vector is flipped to point away from the target.

Overshooting

The **seek** behaviour can lead to overshooting:



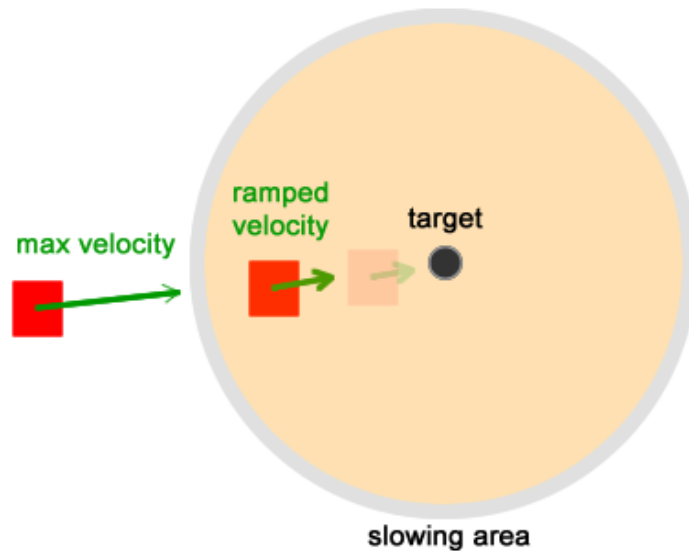
In 2D, this looks like orbiting:



Arrive

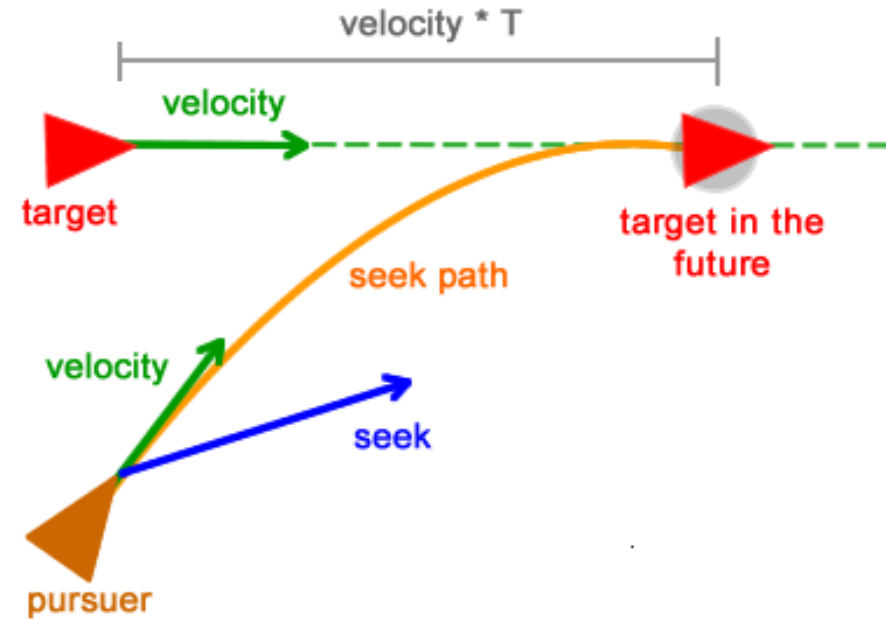
Idea: Modify **seek** to slow down once within a certain distance of the target to avoid overshooting.

- Let d be the distance of the character from the target.
- Let r be the radius of a circle centred on the target.
- If $d \geq r$, don't adjust the desired velocity (just use **seek**).
- If $d < r$, scale seek's desired velocity by d/r .
- As the character approaches the target, the desired velocity approaches zero.



Pursuit and Evasion

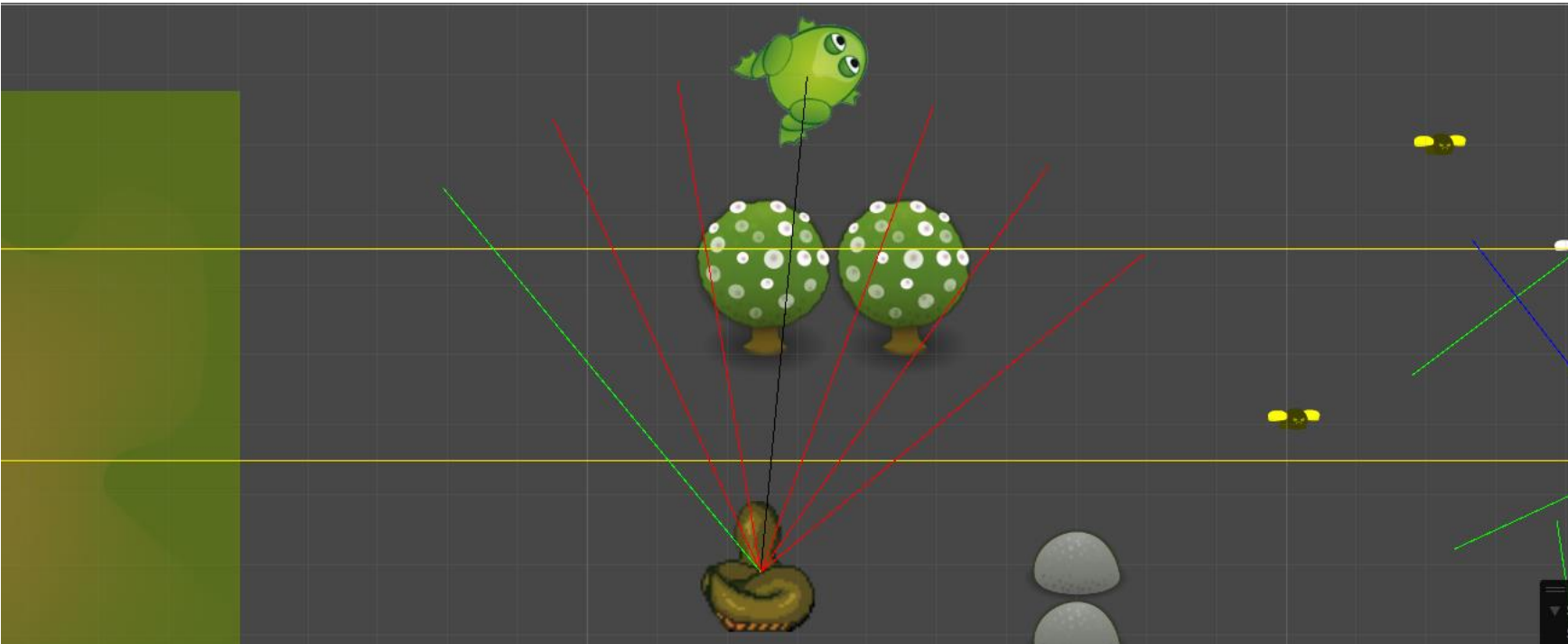
- **Pursuit** is similar to **seek** except that the target is another moving character.
 - Assume that the target will travel at constant velocity during the prediction interval T .
 - Target's future position is estimated by multiplying its velocity by T and adding that offset to its current position.
- **Evasion** is the same idea, but applied to **flee**.
- **Question:** What could go wrong?



Obstacle Avoidance

There are lot of different ways of implementing obstacle avoidance.
(Try Googling “obstacle avoidance steering behavior”, and you will see what I mean.)

For Assignment 1, you are expected to use a specific method though (see next slides).



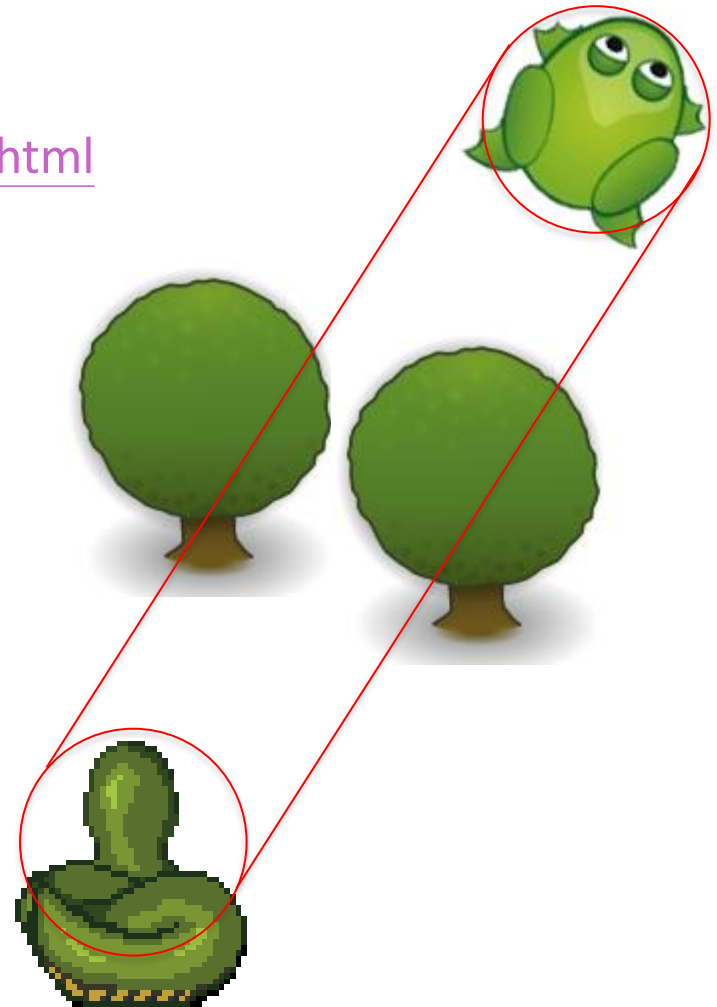
Obstacle Avoidance

Step 1: Fire a circle cast at the target.

If it doesn't hit anything, just go straight to the target.

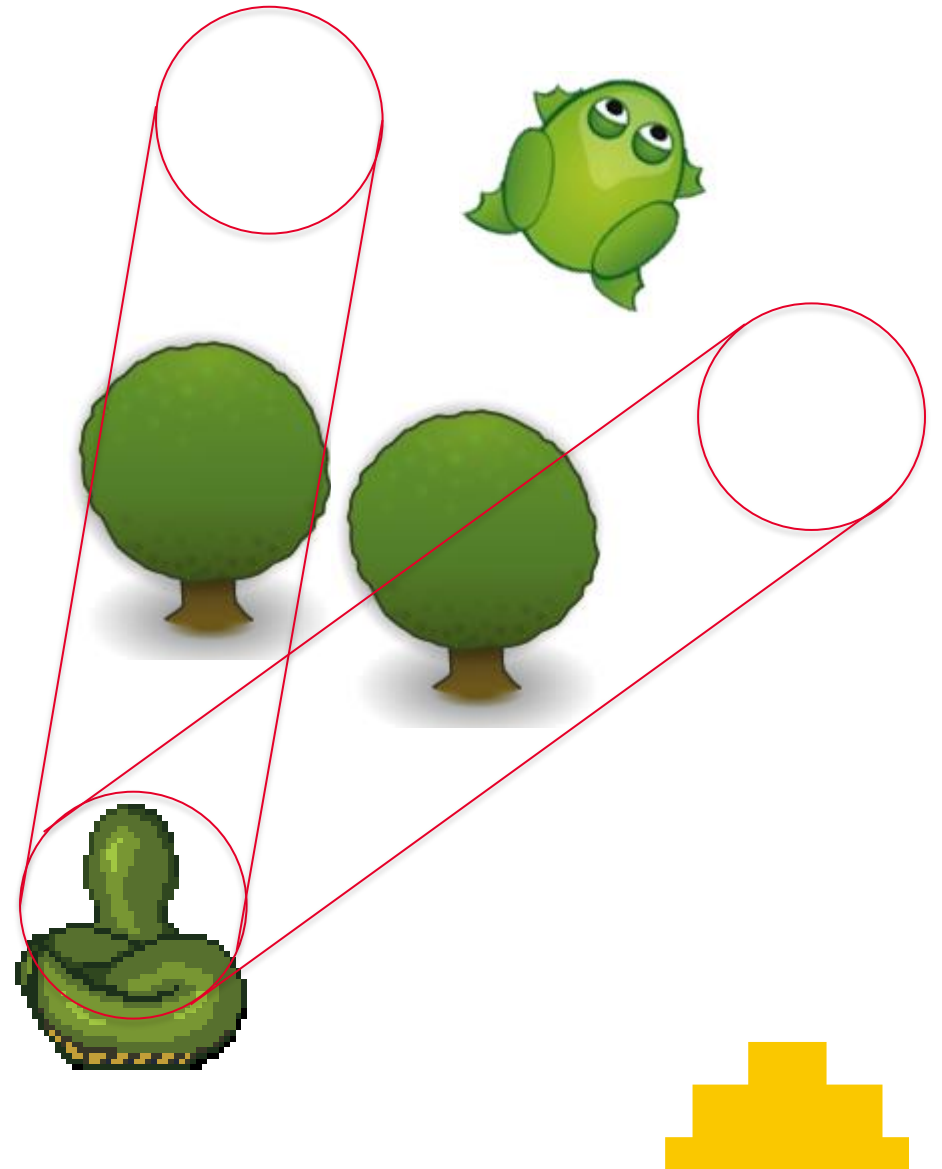
<https://docs.unity3d.com/ScriptReference/Physics2D.CircleCast.html>

Question: Why a circle cast?



Obstacle Avoidance

Step 2: If the original circle cast didn't reveal a free path, fire two new circle casts, one rotated clockwise, one rotated anti-clockwise.

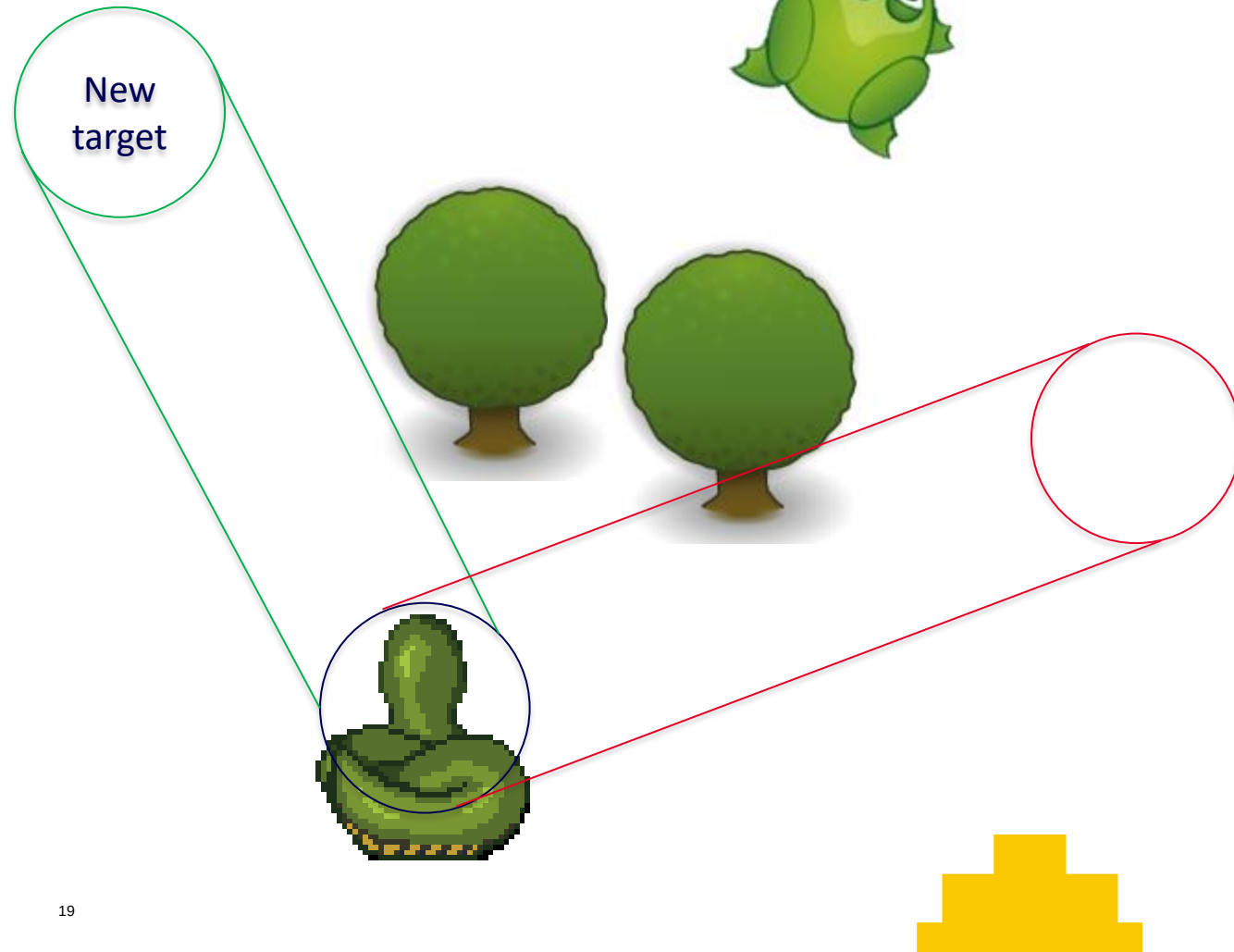


Obstacle Avoidance

Step 3: Keep trying until a free path is found.

Instead of steering towards the original target via seek or arrive, steer towards the endpoint of the free path.

Note: If no free path can be found, there's not much that can be done. Just steer toward the original target.

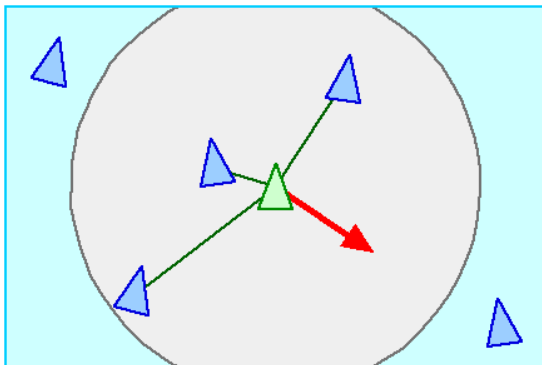


Boids Flocking

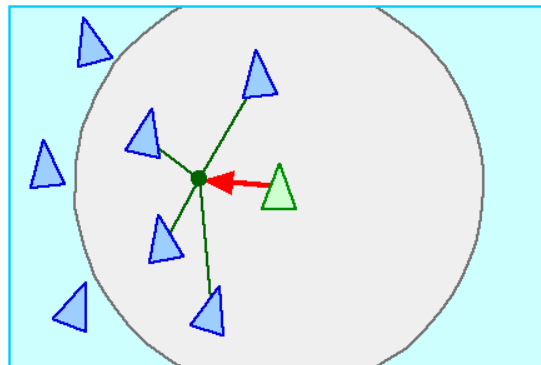
Boids flocking is an emergent behaviour arising from three underlying behaviours: **separation**, **cohesion**, and **alignment**.

It is used to create realistic-looking flocks/swarms.

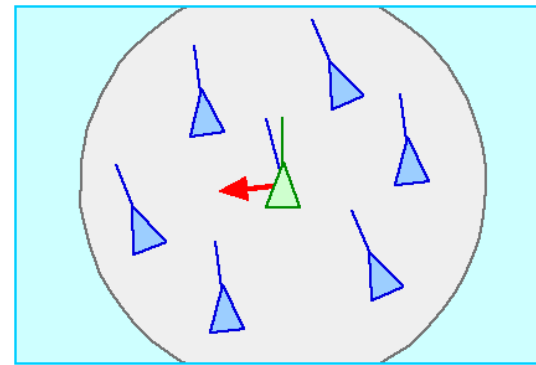
Each character reacts to other characters in its local neighbourhood.



Separation

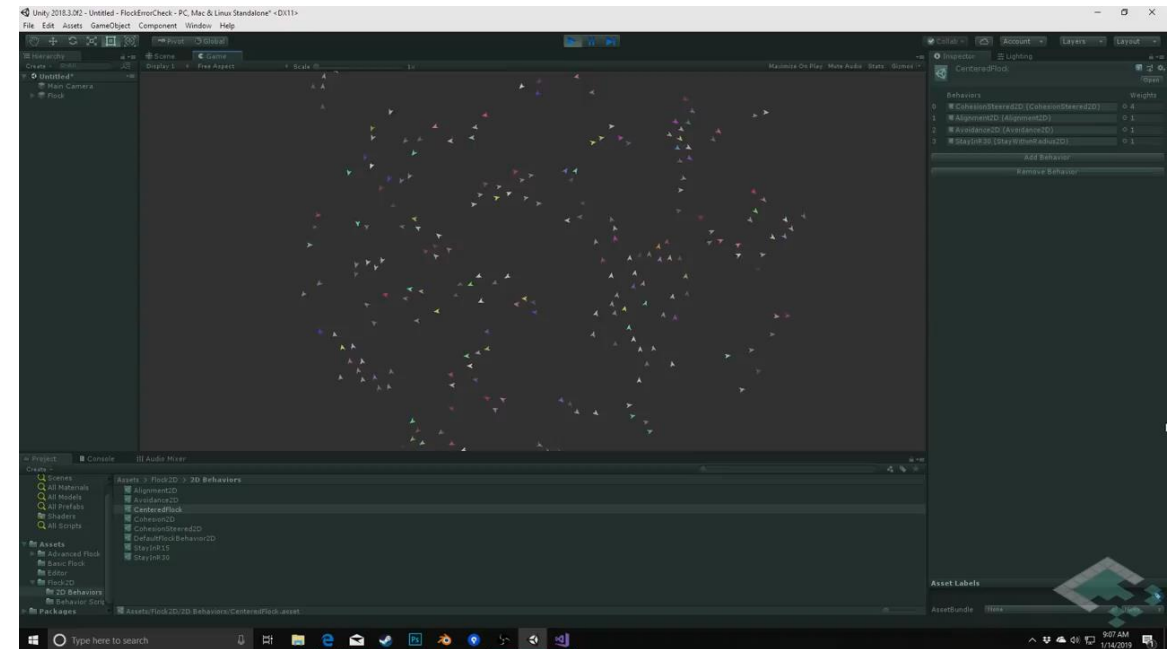


Cohesion



Alignment

The screenshot displays the Unity game engine interface. The main view is a 2D top-down perspective of a game environment. The environment is a dark gray square with a white grid. A white character is positioned in the center of the grid. Surrounding the character are several small white dots, which represent obstacles or targets. The interface includes a top toolbar with icons for play, pause, and other functions. On the right side, there is a panel with a 'Hierarchy' tab, which shows a list of objects in the scene. The list includes 'AI Navigation', 'Surfaces', 'Agents', and 'Obstacles'. The 'Agents' section is expanded, showing a list of agents with their names and positions. The 'Obstacles' section is also expanded, showing a list of obstacles with their names and positions.



Boids Flocking: Separation

A repulsion force away from each of the character's N neighbours.

Calculate the desired velocity, $\mathbf{s}$, as follows:

- Let $\mathbf{p}$ denote the character's position.
- Let $\mathbf{n}_i$ denote the i th neighbour's position, $i \in 1 \dots N$.
- Let d_i = distance from $\mathbf{p}$ to $\mathbf{n}_i$

- $$\mathbf{s}_{raw} = \sum_{i=1}^N \frac{\mathbf{p} - \mathbf{n}_i}{d_i^2}$$

- $$\mathbf{s} = \frac{\mathbf{s}_{raw}}{|\mathbf{s}_{raw}|} * maxSpeed$$

If there are no neighbours, set $\mathbf{s}$ to zero.

Note: This is equivalent to:

$$\mathbf{s}_{raw} = \sum_{i=1}^N \frac{\mathbf{u}_i}{d_i}$$

where $\mathbf{u}_i$ is a unit vector in the direction of $\mathbf{p} - \mathbf{n}_i$

As we did for **seek**, scale the magnitude of the desired velocity to *maxSpeed*.

Boids Flocking: Cohesion

An attractive force towards the average position of the character's neighbours.

Calculate the desired velocity, $\mathbf{c}$, as follows:

- Let $\mathbf{p}$ denote the character's position.
- Let $\mathbf{n}_i$ denote the i th neighbour's position, $i \in 1 \dots N$.

- $$\mathbf{n}_{ave} = \frac{\sum_{i=1}^N \mathbf{n}_i}{N}$$

- $$\mathbf{c}_{raw} = \mathbf{n}_{ave} - \mathbf{p}$$

- $$\mathbf{c} = \frac{\mathbf{c}_{raw}}{|\mathbf{c}_{raw}|} * maxSpeed$$

If there are no neighbours, set $\mathbf{c}$ to zero.

Boids Flocking: Alignment

Steers the character in the direction of the average velocity of its neighbours.

Calculate the desired velocity, $\mathbf{a}$, as follows:

- Let $\mathbf{v}_i$ denote the i th neighbour's velocity, $i \in 1 \dots N$.

- $$\mathbf{v}_{ave} = \frac{\sum_{i=1}^N \mathbf{v}_i}{N}$$

- $$\mathbf{a} = \frac{\mathbf{v}_{ave}}{|\mathbf{v}_{ave}|} * maxSpeed$$

If there are no neighbours, set $\mathbf{a}$ to zero.

Boids Flocking: Putting it All Together

The overall desired velocity is calculated as a weighted sum of the separation, cohesion and alignment components:

- Let w_s , w_c and w_a denote the separation, cohesion and alignment weights, respectively.
- $\mathbf{desiredVel}_{raw} = w_s \mathbf{s} + w_c \mathbf{c} + w_a \mathbf{a}$
- $\mathbf{desiredVel} = \frac{\mathbf{desiredVel}_{raw}}{|\mathbf{desiredVel}_{raw}|} * maxSpeed$

Note: It might seem strange that we normalise to *maxSpeed* twice (once for each component individually, then again at the end), but this makes balancing the weights a little easier.

Not all implementations follow this approach.

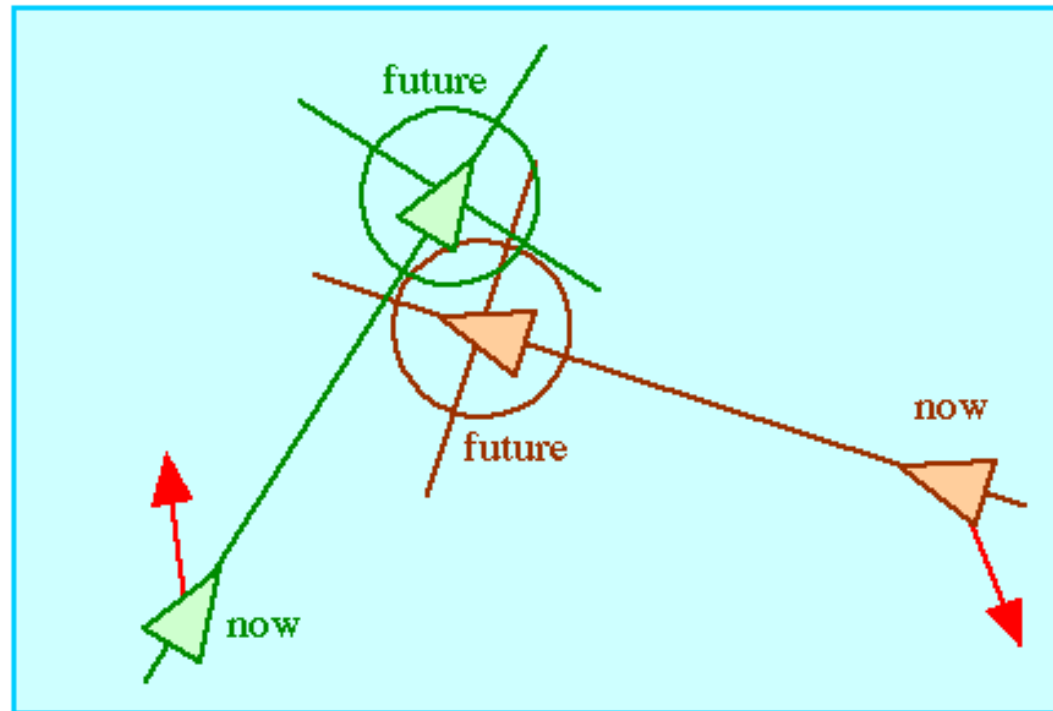
Extensions

(Additional things, not required for the assignment)

Collision Avoidance with Moving Obstacles

Similar to **pursuit** and **evasion**, obstacle avoidance can be adapted to handle moving obstacles.

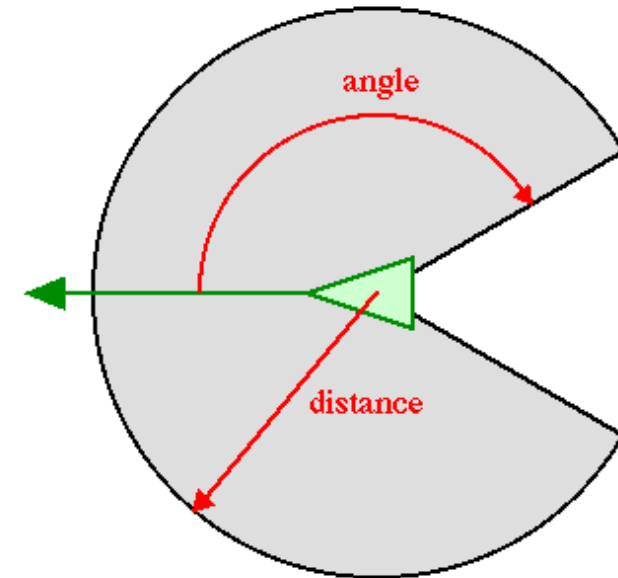
If the moving obstacle corresponds to a character, this is a tricky, multi-agent problem!



Boids Flocking: Neighbourhood

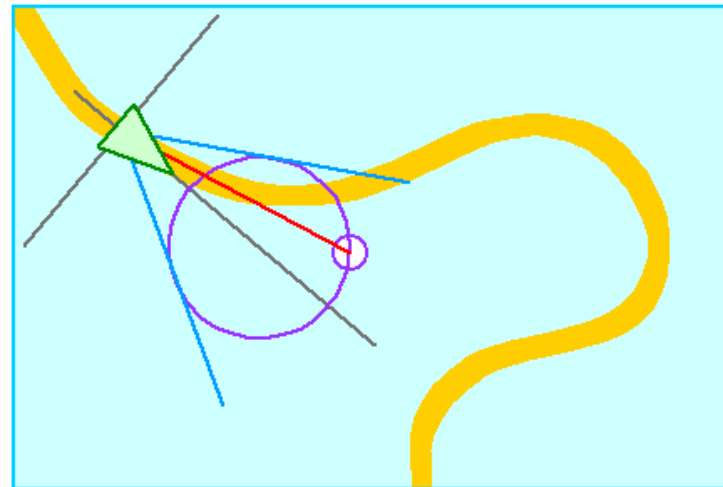
For boids flocking, the “neighbours” of the character are typically defined as all characters within a certain distance.

This can be made more realistic by assuming that the character has a limited field of view. Characters that are outside the field of view are not treated as neighbours.



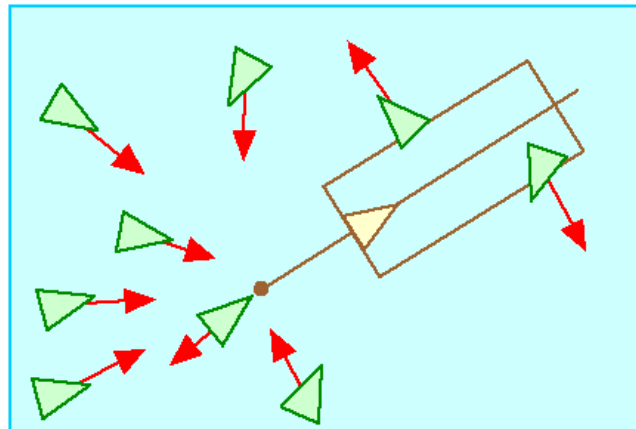
Wander

- Aim is to mimic realistic “casual” movement via random steering.
- However, uniform random velocity changes don’t look realistic.
- Better approach: Imagine a circle in front of the character and steer to a random point on the circle radius.
- The circle size controls the amount of randomness.
- See <https://gamedevelopment.tutsplus.com/tutorials/understanding-steering-behaviors-wander--gamedev-1624>



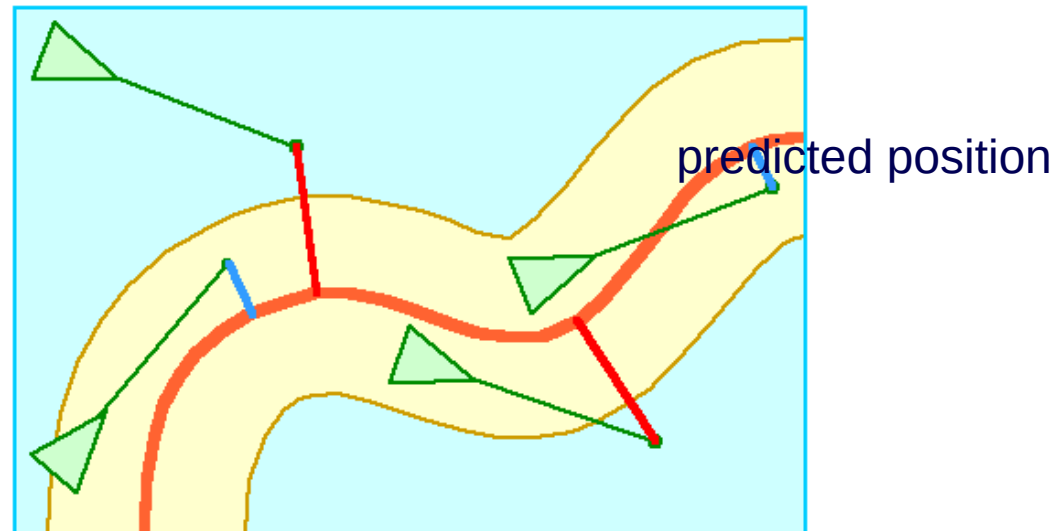
Leader Following

- **Leader following** can be used to mimic scenarios such a flock of sheep following a shepherd.
- If a follower finds itself in a rectangular region in front of the leader, it will steer laterally away from the leader's path.
- Otherwise, the arrival target is a point offset slightly behind the leader.
- The followers use separation behavior to prevent crowding.
- See <https://gamedevelopment.tutsplus.com/tutorials/understanding-steering-behaviors-leader-following--gamedev-10810>



Path following

- Aim is to move a character along the path while staying within the specified radius of the spine.
- If the projection distance is less than the path radius, no corrective steering is required.
- Otherwise, seek towards the on-path projection of the predicted future position.
- See <https://gamedevelopment.tutsplus.com/tutorials/understanding-steering-behaviors-path-following--gamedev-8769>



Combining Steering Behaviors

- Behaviours can be combined in two main ways:
 - Blending
 - Switching
- **Blending:** Compute each of the steering forces separately then sum them together, possibly with different weighting factors. (This is how flocking works.)
 - Can be tricky, since the forces may cancel out or yield unintelligent behaviour when combined.
- **Switching:** Toggle between different steering behaviours, depending on the current state of the world.
 - Next week we'll look at finite state machines (FSMs), which can be used to determine when to switch between behaviours.

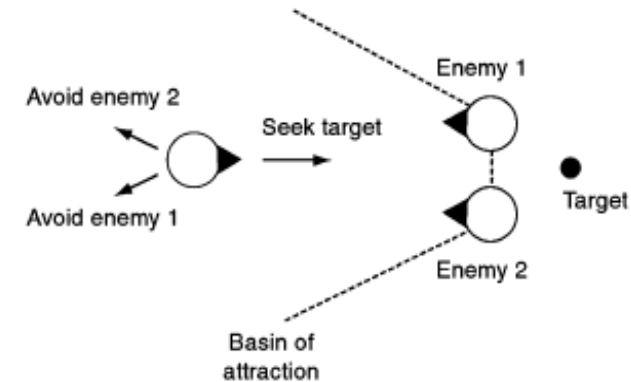


Figure 3.34 A stable equilibrium

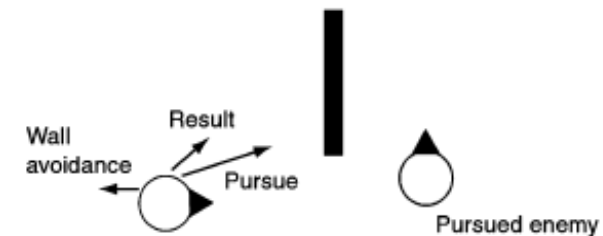


Figure 3.35 Can't avoid an obstacle and chase